

AI Plant Diagnosis System

React Native Frontend | PredictionScreen.js | Final Year Project

```
import React, { useState, useRef, useEffect } from "react";
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  Image,
  ScrollView,
  ActivityIndicator,
  TextInput,
  ImageBackground,
  Animated,
  Platform
} from "react-native";
import * as ImagePicker from "expo-image-picker";
import axios from "axios";
import { useTranslation } from 'react-i18next';
import { BlurView } from "expo-blur";

export default function PredictionScreen() {
  const { t } = useTranslation();
  const [selectedImage, setSelectedImage] = useState(null);
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [acres, setAcres] = useState("1");

  // animation ref for cards
  const fadeAnim = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    // animate on mount
    Animated.timing(fadeAnim, {
      toValue: 1,
      duration: 700,
      useNativeDriver: true,
    }).start();
  }, []);

  const calculateIncome = (incomeStr, acreValue) => {
    const numericIncome = parseFloat(incomeStr.replace(/^[^0-9.]/g, ''));
    if (isNaN(numericIncome)) return "N/A";
    return "Rs." + (numericIncome * parseFloat(acreValue)).toLocaleString();
  };

  const calculateYield = (yieldStr, acreValue) => {
    const matches = yieldStr.match(/(\d+)/g);
    if (!matches) return yieldStr;

    const min = parseFloat(matches[0]) * parseFloat(acreValue);
    const max = matches[1] ? parseFloat(matches[1]) * parseFloat(acreValue) : null;

    const unit = yieldStr.includes("tons") ? "tons" : "quintals";
    return max ? `${min}-${max} ${unit}` : `${min} ${unit}`;
  };

  const pickImage = async () => {
    const permission = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (!permission.granted) return alert("Permission required!");

    const img = await ImagePicker.launchCameraAsync({
      // FIX 1: Replaced deprecated MediaTypeOptions.Images
      mediaTypes: ImagePicker.MediaType.IMAGE,
    });
  };
}
```

```

        allowsEditing: false,
        quality: 1,
        exif: false,
    });

    if (!img.canceled) {
        setSelectedImage(img.assets[0].uri);
        setResult(null);
    }
};

const captureImage = async () => {
    const permission = await ImagePicker.requestCameraPermissionsAsync();
    if (!permission.granted) return alert("Camera permission required!");

    const img = await ImagePicker.launchCameraAsync({
        mediaTypes: ImagePicker.MediaTypeOptions.Images,
        allowsEditing: false,
        quality: 1,
        exif: false,
    });

    if (!img.canceled) {
        setSelectedImage(img.assets[0].uri);
        setResult(null);
    }
};

const uploadImage = async () => {
    if (!selectedImage) return alert("Select an image first!");
    if (!acres || isNaN(acres) || parseFloat(acres) <= 0) return alert("Please enter a valid land area in acres.");

    setLoading(true);
    setResult(null);

    const formData = new FormData();
    formData.append("image", {
        uri: selectedImage,
        type: "image/jpeg",
        name: "photo.jpg",
    });

    try {
        const res = await axios.post(
            "http://10.30.65.139:8081/predict",
            formData,
            {
                headers: { "Content-Type": "multipart/form-data" },
            }
        );

        setResult(res.data);
    } catch (error) {
        console.log(error);
        alert("Error connecting to server");
    }

    setLoading(false);
};

return (
    <ImageBackground
        source={require("../assets/leaf2.png")}
        style={{ flex: 1 }}
        resizeMode="cover"
    >
        {/* dark gradient overlay to increase contrast */}
        <View style={styles.overlay} />
    </ImageBackground>

```

```

<ScrollView style={styles.container} contentContainerStyle={styles.contentContainer}>
  <Animated.View style={[styles.glassCard, { opacity: fadeAnim, transform: [{ translateY: fadeAnim.interpolate(
    <BlurView intensity={60} tint="default" style={styles.innerBlur}>

    <Text style={styles.title}>{t('pred_title')}</Text>

    <View style={styles.inputContainer}>
      <Text style={styles.label}>{t('enter_acres')}</Text>
      <TextInput
        style={styles.input}
        keyboardType="numeric"
        value={acres}
        onChangeText={setAcres}
        placeholder="e.g. 2.5"
        placeholderTextColor="rgba(0,0,0,0.45)"
      />
    </View>

    <View style={styles.buttonRow}>
      <TouchableOpacity style={styles.button} onPress={pickImage}>
        <Text style={styles.buttonText}>{t('pick_image')}</Text>
      </TouchableOpacity>

      <TouchableOpacity style={styles.button} onPress={captureImage}>
        <Text style={styles.buttonText}>{t('camera')}</Text>
      </TouchableOpacity>
    </View>

    {selectedImage && (
      <View style={styles.previewWrap}>
        <Image source={{ uri: selectedImage }} style={styles.preview} />
        <BlurView intensity={40} tint="light" style={styles.previewBadge}>
          <Text style={styles.previewBadgeText}>Selected</Text>
        </BlurView>
      </View>
    )}

    <TouchableOpacity style={styles.uploadBtn} onPress={uploadImage} activeOpacity={0.85}>
      <Text style={styles.uploadText}>{t('predict_btn')}</Text>
    </TouchableOpacity>

    {loading && <ActivityIndicator size="large" color="#008000" style={{ marginTop: 10 }} />}

    {result && (
      <View style={styles.resultBox}>
        <BlurView intensity={70} tint="light" style={styles.resultBlur}>
          <Text style={styles.resultText}>{t('result_crop')} {result.crop}</Text>
          <Text style={styles.resultText}>{t('result_disease')} {result.disease}</Text>
          <Text style={styles.resultText}>{t('result_conf')} {result.confidence}%</Text>

          {result.diagnosis && (
            <View style={styles.diagnosisBox}>
              <Text style={styles.diagnosisTitle}>{t('dis_desc_title')}</Text>
              <Text style={styles.diagnosisText}>{result.diagnosis}</Text>
            </View>
          )}

          {result.recovery && (
            <View style={styles.recoveryBox}>
              <Text style={styles.diagnosisTitle}>
                {result.disease === 'Healthy' ? t('care_instr') : t('cure_instr')}
              </Text>
              <Text>
                {result.recovery.map((step, index) => (
                  <View key={index} style={styles.stepContainer}>
                    <Text style={styles.stepNumber}>{index + 1}</Text>
                    <Text style={styles.stepText}>{step}</Text>
                  </View>
                ))}
              </Text>
            </View>
          )}
        </BlurView>
      </View>
    )}
  </Animated.View>
</ScrollView>

```

```

        </View>
      )}

      {result.yield_impact && (result.disease !== 'Healthy') && (
        <View style={styles.yieldBox}>
          <Text style={styles.diagnosisTitle}>{t('yield_est_title')}

```

```

    marginHorizontal: 10,
    overflow: "hidden",

    // softer shadow
    shadowColor: "#000",
    shadowOpacity: 0.22,
    shadowRadius: 14,
    shadowOffset: { width: 0, height: 6 },
    elevation: 12,
  },

  // inner blur layer
  innerBlur: {
    padding: 18,

    // Less brightness
    backgroundColor: "rgba(255,255,255,0.04)",

    // Softer border
    borderWidth: 1,
    borderColor: "rgba(255, 255, 255, 0.09)",
  },

  title: {
    fontSize: 26,
    fontWeight: "800",
    textAlign: "center",
    marginBottom: 12,
    color: "#fff",
    textShadowColor: "rgba(0,0,0,0.3)",
    textShadowOffset: { width: 0, height: 2 },
    textShadowRadius: 6,
  },

  inputContainer: {
    marginBottom: 12,
  },
  label: {
    fontSize: 14,
    fontWeight: "700",
    color: "#f5f5f5ff",
    marginBottom: 6,
  },

  input: {
    borderWidth: 0,
    backgroundColor: "rgba(255, 255, 255, 0.48)",
    borderRadius: 10,
    padding: 12,
    fontSize: 16,
    color: "#131212ff",
  },

  buttonRow: {
    flexDirection: "row",
    justifyContent: "space-between",
    marginTop: 10,
  },

  button: {
    flex: 1,
    backgroundColor: "rgba(33, 110, 33, 0.9)",
    padding: 12,
    borderRadius: 10,
    marginHorizontal: 6,
    alignItems: "center",
    borderWidth: 1,
    borderColor: "rgba(255,255,255,0.14)",
  },

```

```

        shadowColor: "#000",
        shadowOpacity: 0.2,
        shadowRadius: 8,
        elevation: 6,
    },

    buttonText: {
        color: "#fff",
        fontWeight: "700",
    },

    previewWrap: {
        marginTop: 14,
        borderRadius: 12,
        overflow: "hidden",
        position: "relative",
        shadowColor: "#000",
        shadowOpacity: 0.35,
        shadowRadius: 12,
        elevation: 10,
    },

    preview: {
        width: "100%",
        height: 260,
    },

    previewBadge: {
        position: "absolute",
        right: 10,
        top: 10,
        paddingHorizontal: 10,
        paddingVertical: 6,
        borderRadius: 999,
        backgroundColor: "rgba(255,255,255,0.18)",
        borderWidth: 1,
        borderColor: "rgba(255,255,255,0.28)",
    },

    previewBadgeText: {
        color: "#0b4b0b",
        fontWeight: "700",
    },

    uploadBtn: {
        marginTop: 14,
        backgroundColor: "rgba(6,80,6,0.95)",
        padding: 14,
        borderRadius: 12,
        alignItems: "center",
        borderWidth: 1,
        borderColor: "rgba(255,255,255,0.16)",
        shadowColor: "#07300729",
        shadowOpacity: 0.28,
        shadowRadius: 12,
        elevation: 8,
    },

    uploadText: {
        color: "#fff",
        fontSize: 18,
        fontWeight: "800",
    },

    resultBox: {
        marginTop: 16,
        borderRadius: 12,
        overflow: "hidden",

```

```
    },

    resultBlur: {
      padding: 14,
      backgroundColor: "rgba(255,255,255,0.06)",
      borderWidth: 1,
      borderColor: "rgba(255,255,255,0.09)",
    },

    resultText: {
      fontSize: 18,
      fontWeight: "700",
      color: "#fff",
      marginBottom: 8,
    },

    diagnosisBox: {
      marginTop: 8,
      borderTopWidth: 1,
      borderTopColor: "rgba(0,0,0,0.06)",
      paddingTop: 10,
    },

    diagnosisTitle: {
      fontSize: 16,
      fontWeight: "800",
      color: "#eef6ef",
      marginBottom: 6,
    },

    diagnosisText: {
      color: "#f2f6f2",
      lineHeight: 20,
    },

    recoveryBox: {
      marginTop: 8,
    },

    stepContainer: {
      flexDirection: "row",
      marginBottom: 6,
      alignItems: "flex-start",
    },

    stepNumber: {
      fontWeight: "bold",
      color: "#cde9c7",
      marginRight: 8,
    },

    stepText: {
      color: "#f1f5f1",
      flex: 1,
      lineHeight: 20,
    },

    yieldBox: {
      marginTop: 12,
    },

    yieldRow: {
      flexDirection: "row",
      justifyContent: "space-between",
    },

    yieldCol: {
      width: "48%",
```

```

        padding: 10,
        borderRadius: 8,
        borderWidth: 1,
        borderColor: "rgba(255,255,255,0.06)",
    },

    earlyCol: {
        backgroundColor: "rgba(220,255,245,0.06)",
    },

    lateCol: {
        backgroundColor: "rgba(255,230,230,0.06)",
    },

    yieldHeader: {
        fontWeight: "700",
        textAlign: "center",
        color: "#fff",
        marginBottom: 8,
    },

    yieldLabel: {
        fontSize: 12,
        color: "#e8f5e8",
        marginTop: 4,
    },

    yieldValue: {
        fontSize: 14,
        fontWeight: "800",
        color: "#ffffff7a",
        marginBottom: 4,
    },

    disclaimer: {
        fontSize: 11,
        color: "#dbe9db9c",
        marginTop: 8,
        textAlign: "right",
        fontStyle: "italic",
    },
});

```